

Plan de implementare AutoPro Daune

AutoPro Daune este o **platformă digitală** care simplifică procesul de raportare și gestionare a daunelor auto. MVP-ul inițial include un **landing page** cu formular de lead-uri și un flux automat de preluare a cererilor. Vom urmări aceste specificații, adăugând apoi treptat funcționalități noi conform celor 5 faze.

Faza 1: Foundation Fixes

În faza de bază asigurăm infrastructura și configurările inițiale: **mediul de dezvoltare, variabile de configurare, schemă de bază de date**, CORS, Redis, etc. De exemplu, vom încărca variabilele din fișierul `.env` folosind `python-dotenv`, vom configura conexiunea la Supabase (schema importată din `supabase_schema.sql`) și vom porni serviciile de Redis pentru caching/deborșare. În FastAPI, corectăm setările CORS din `main.py` și importăm modulele necesare.

- **Cod Python (FastAPI):** Deși endpoint-ul de sănătate (`/health`) există, putem adăuga un endpoint de test simplu. De exemplu, în `backend/routes/health.py` putem avea:

```
@router.get("/ping")
async def ping() -> Dict[str, Any]:
    """Endpoint de test rapid pentru API."""
    return {"status": "alive", "timestamp": datetime.now().isoformat()}
```

De asemenea, verificăm inițializarea conexiunii la baza de date și la Redis în `backend/main.py`, eventual cu tratament de excepție.

- **Cod TypeScript (frontend):** În `autoprotapi.ts`, ne asigurăm că metoda de `healthCheck` returnează starea API-ului. De exemplu:

```
async healthCheck(): Promise<{ status: string }> {
    const response = await axios.get(`${this.baseUrl}/api/health`);
    return response.data;
}
```

Dacă lipsesc alți indicatori, îi adăugăm similar.

- **React (UI):** Creăm un component simplu de **Home** care afișează statusul serverului (ex. „Server Alive”) și eventual un **Dashboard** de administrare minimă. Aceste componente vor apela `healthCheck` și vor afișa rezultatul. De exemplu, `src/components/Home.tsx`:

```
import React, { useEffect, useState } from 'react';
import autoproApi from '../services/autoproApi';

export default function Home() {
  const [status, setStatus] = useState("");

  useEffect(() => {
    autoproApi.healthCheck().then(res => setStatus(res.status));
  }, []);

  return <div>API Status: {status}</div>;
}
```

- **PowerShell (scripts):** Un script `run-dev.ps1` de exemplu poate porni serverul FastAPI și frontend-ul:

```
# Script PowerShell pentru rulare locală
$Env:PYTHONPATH = "backend"
uvicorn backend.main:app --reload --host 0.0.0.0 --port 8000
npm --prefix frontend install
npm --prefix frontend run dev
```

Un alt script `smoke-test.ps1` poate trimite cereri rapide `/health` sau `/ping` pentru verificare, de exemplu:

```
$result = Invoke-RestMethod -Uri http://localhost:8000/api/health
if ($result.status -eq "OK") { Write-Host "API ONLINE" } else { Write-Host
"API DOWN" }
```

Figura: Arhitectura de bază FE ↔ BE ↔ DB (ilustrație generică).

Faza 2: Core Business (Leads, Financiar, Notificări, Video)

Implementăm funcționalitățile principale conform cerințelor: **gestionare lead-uri, facturi și plăți, sistem de notificări, upload și procesare video**.

- **Lead-uri (FastAPI):** În `backend/routes/leads.py` deja există rutare pentru `GET /leads` și `POST /leads`, dar putem adăuga endpoint-urile lipsă (ex. `PUT /leads/{id}`, `DELETE /leads/{id}`, `POST /leads/{id}/score`). Exemplu de endpoint de scorare automată:

```
@router.post("/score")
async def score_lead(lead: LeadDT0) -> Dict[str, Any]:
```

```

"""Calculează scorul unui lead nou."""
points = calculate_lead_score(lead) # presupunem funcție ajutătoare
return {"lead_id": lead.id, "score": points}

```

- **Financiar:** În `backend/routes/financial.py` adăugăm, de exemplu, endpoint-ul de raport financiar:

```

@router.get("/summary")
async def get_financial_summary() -> Dict[str, Any]:
    """Returnează total venituri și cheltuieli din BD."""
    supabase = get_supabase_service_instance()
    incomes = supabase.query("payments").sum("amount").execute()
    expenses = supabase.query("invoices").sum("total").execute()
    return {"total_income": incomes, "total_expenses": expenses}

```

În frontend, în `autoproApi.ts` implementăm metodele corespunzătoare:

```

async getFinancialSummary(): Promise<{ total_income: number,
total_expenses: number }> {
    const res = await axios.get(`${this.baseUrl}/api/financial/summary`);
    return res.data;
}

```

- **Notificări:** Observăm că în `backend/routes/notifications.py` există o TODO la trimiterea email-urilor. Îl implementăm, de exemplu:

```

import smtplib
@router.post("/email")
async def send_email(recipient: str = Form(...), subject: str = Form(...),
content: str = Form(...)) -> Dict[str, Any]:
    """Trimite un email (SMTP simplu ca exemplu)."""
    try:
        smtp = smtplib.SMTP(os.getenv("SMTP_HOST"),
int(os.getenv("SMTP_PORT")))
        smtp.starttls()
        smtp.login(os.getenv("SMTP_USER"), os.getenv("SMTP_PASSWORD"))
        msg = f"From: AutoPro Daune <{os.getenv('SMTP_USER')}>\r\nTo:
{recipient}\r\nSubject: {subject}\r\n\r\n{content}"
        smtp.sendmail(os.getenv("SMTP_USER"), recipient,
msg.encode('utf-8'))
        smtp.quit()
        return {"success": True, "message": "Email trimis cu succes."}
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Eroare email: {e}")

```

În `autoproApi.ts` adăugăm metoda corespunzătoare:

```
async sendEmail(recipient: string, subject: string, content: string):
Promise<any> {
  return axios.post(`${this.baseUrl}/api/notify/email`, { recipient,
subject, content })
    .then(res => res.data);
}
```

- **Video:** În `backend/routes/video.py` implementăm un endpoint simplu de încărcare pentru generare video (stub). De exemplu:

```
@router.post("/simple-generate", status_code=202)
async def simple_generate(video_data: ManoleGenerateRequest) -> Dict[str,
Any]:
    """Adaugă un job simplu de generare video în coadă."""
    supabase = get_supabase_service_instance()
    job_id = supabase.generate_uuid()
    supabase.table("video_jobs").insert({
        "job_id": job_id,
        "status": JobStatus.queued.value,
        **video_data.dict()
    }).execute()
    return {"message": "Job adăugat la coadă", "job_id": job_id}
```

În frontend, adăugăm în `autoproApi.ts` metoda `generateSimpleVideo`:

```
async generateSimpleVideo(data: VideoRequest): Promise<{ job_id: string }>
{
  const res = await axios.post(`${this.baseUrl}/api/video/simple-
generate`, data);
  return res.data;
}
```

- **React (UI):** Se creează componente noi: de exemplu `LeadList.tsx` pentru afișarea lead-urilor (schema separată, apelează `getLeads()` și stochează în state), `InvoiceForm.tsx` pentru crearea de facturi, `NotificationPanel.tsx` pentru listarea notificărilor primite și opțiunea de „mark as read”. În aceste componente vom folosi Hook-uri React pentru stat și/sau React Context (SRP: un component afișează datele, alt component se ocupă de logică). Exemplu:

```
// src/components/LeadList.tsx
import React, { useEffect, useState } from 'react';
import autoproApi, { Lead } from '../services/autoproApi';

export function LeadList() {
```

```
const [leads, setLeads] = useState<Lead[]>([]);
useEffect(() => {
  autoproApi.getLeads().then(data => setLeads(data));
}, []);
return (
  <div>
    <h2>Lista lead-uri</h2>
    <ul>
      {leads.map(l => <li key={l.id}>{l.name} - {l.phone}</li>)}
    </ul>
  </div>
);
}
```

- **PowerShell:** Scripturi pentru testare rapidă a noilor funcționalități, de exemplu un

`test_leads.ps1` care trimite un lead prin curl:

```
$lead = @{ name="Ion Popescu"; phone="0712345678"; details="Accident
minor"; photos=@() } | ConvertTo-Json
$res = Invoke-RestMethod -Uri http://localhost:8000/api/leads -Method Post
-Body $lead -ContentType "application/json"
Write-Host "Lead creat: ID" $res.id
```

Sau un script `test_notifications.ps1`:

```
Invoke-RestMethod http://localhost:8000/api/notify/test -Body
@{message="Salut din test"} -ContentType application/json
```

Figura: Exemplu de flux principal FE-BE-DB (simplificat).

Faza 3: Growth Features (Growth Engine, Conversion, Nurturing, Affiliate)

Adăugăm modulele de creștere și viralizare: **growth engine** (campanii de marketing), **customer nurturing**, **conversion optimization**, **affiliate program**. Majoritatea au rute prestabilite în backend, dar multe erau doar scheme. Vom scrie logica aferentă.

- **Growth Engine:** În `backend/routes/growth_engine.py` se poate introduce, de exemplu, un endpoint care pornește o campanie de email marketing automat:

```
@router.post("/launch-campaign")
async def launch_campaign(name: str = Query(...)) -> Dict[str, Any]:
    """Inițiază o campanie de generare de lead-uri."""
    campaign_id =
```

```
get_supabase_service_instance().launch_marketing_campaign(name)
    return {"status": "started", "campaign_id": campaign_id}
```

În frontend, metoda aferentă în `autoproApi.ts`:

```
async launchCampaign(name: string): Promise<{ campaign_id: string }> {
    const res = await axios.post(`${this.baseUrl}/api/growth-engine/launch-campaign?name=${encodeURIComponent(name)}`);
    return res.data;
}
```

- **Conversion & Nurturing:** De exemplu, `backend/routes/conversion.py` ar putea conține endpoint-uri ce înregistrează conversii sau email-uri de tip nurturing:

```
@router.post("/track")
async def track_conversion(lead_id: str = Form(...)) -> Dict[str, Any]:
    """Înregistrează conversia unui lead."""
    supabase = get_supabase_service_instance()
    supabase.table("leads").update({"converted": True}).eq("id",
lead_id).execute()
    return {"message": f"Lead {lead_id} marcat conversie"}
```

În frontend, adăugăm:

```
async markLeadConverted(leadId: string): Promise<any> {
    return axios.post(`${this.baseUrl}/api/conversion/track`, { lead_id:
leadId });
}
```

- **Affiliate:** În `backend/routes/affiliate_multiplication.py` deja există logica de creare cont de affiliate. Putem completa endpoint-urile lipsă, de ex. pentru listare/stare:

```
@router.get("/status/{affiliate_id}")
async def get_affiliate_status(affiliate_id: str) -> Dict[str, Any]:
    """Returnează statusul și comisioanele unui afiliat."""
    supabase = get_supabase_service_instance()
    data = supabase.table("affiliates").select().eq("affiliate_id",
affiliate_id).execute()
    if data:
        return data[0]
    raise HTTPException(status_code=404, detail="Afiliat negăsit")
```

În `autoproApi.ts`:

```

async getAffiliateStatus(id: string): Promise<any> {
  const res = await axios.get(`${this.baseUrl}/api/affiliate-
multiplication/status/${id}`);
  return res.data;
}

```

- **React:** Creăm componente pentru aceste funcționalități de creștere, de exemplu un `AffiliateDashboard.tsx` care afișează comisioanele și statusul afiliatului curent (apelând `getAffiliateStatus`). Un `CampaignForm.tsx` permite lansarea de campanii noi, etc. Fiecare componentă va avea la bază SRP și va utiliza hooks pentru date și efecte.
- **PowerShell:** Scripturi pentru testarea noilor endpoint-uri, de exemplu `test_affiliate.ps1` cu:

```

$payload = @{affiliate_id="AFF123"; name="Marian"; email="marian@x.com";
phone="0712345678"} | ConvertTo-Json
$res = Invoke-RestMethod http://localhost:8000/api/affiliate-
multiplication/create-affiliate -Method Post -Body $payload -ContentType
'application/json'
Write-Host "Afiliat creat: $($res.affiliate_details.affiliate_code)"

```

Faza 4: Advanced & AI (Video avansat, Analytics, Monitoring)

Extindem funcționalități avansate:

- **Video avansat:** Rutele pentru „professional video” și „advanced video” erau definite (`backend/routes/professional_video.py`, etc.). Vom adăuga logica de generare video (de exemplu, folosind servicii AI). Exemplu de endpoint:

```

@router.post("/avatar-video", status_code=202)
async def generate_avatar_video(request: ManoleGenerateRequest) ->
Dict[str, Any]:
    """Generează videoclip avansat cu avatar."""
    job_id = process_avatar_video(request) # funcție ipotetică
    return {"status": "queued", "job_id": job_id}

```

În `autoproApi.ts`, adăugăm metoda:

```

async generateAvatarVideo(data: AvatarVideoRequest): Promise<any> {
  const res = await axios.post(`${this.baseUrl}/api/professional-video/
avatar-video`, data);
  return res.data;
}

```

- **Analytics:** În `backend/routes/growth_analytics.py` implementăm rularea unor rapoarte (ex. Google Analytics sau interogare BD). Exemplu:

```
@router.get("/overview")
async def get_analytics_overview() -> Dict[str, Any]:
    """Rapoarte de trafic și conversii."""
    # Exemplu: colectare date
    total_leads =
    get_supabase_service_instance().table("leads").select("id",
count="exact").execute()
    total_sales =
    get_supabase_service_instance().table("leads").select("converted",
count="exact").eq("converted", True).execute()
    return {"total_leads": total_leads, "total_sales": total_sales}
```

În `autoproApi.ts`:

```
async getAnalyticsOverview(): Promise<any> {
    const res = await axios.get(`${this.baseUrl}/api/growth-analytics/
overview`);
    return res.data;
}
```

- **Monitoring:** În `backend/routes/logs.py` putem expune endpoint-ul de colectare log-uri:

```
@router.get("/")
async def get_logs() -> List[str]:
    """Returnează ultimele intrări din log."""
    logs = []
    with open("/app/logs/app.log") as f: logs = f.readlines()[-50:]
    return logs
```

În frontend, un component `MonitoringDashboard.tsx` poate afișa aceste mesaje în timp real.

- **React:** Creăm pagini avansate, cum ar fi un tab **Analytics** cu grafice (de ex. folosind Chart.js) pentru datele returnate de API. Un component **VideoGenerator.tsx** permite încărcarea imaginilor și textului pentru generare video AI, afișând progresul job-ului.

Faza 5: Frontend + API Integration & Testing

În faza finală integrăm totul la nivel de UI/UX, legând componentele React de API-ul din backend și scriem teste de sănătate QA.

- **Componente React finale:** Verificăm că fiecare endpoint are un component corespunzător și stocare de stare clar definită. Exemplu: `LeadsDashboard.tsx` importă `LeadList` și eventual un formular de căutare filtru, `FinancialReport.tsx` consumă datele din `getFinancialSummary`,

`Notifications.tsx` afișează notificările (`getNotifications`) și permite marcare ca citit (`markNotificationRead`). Urmând principiul SRP, logica de apel API poate fi într-un custom hook (ex. `useLeads`) iar UI-ul într-un component separată.

- **Metodele lipsă în** `autoproApi.ts`: Verificăm și completăm orice metodă lipsă conform endpoint-urilor create. De exemplu, dacă am adăugat `/api/logs` în backend, adăugăm în `autoproApi.ts`:

```
async getLogs(): Promise<string[]> {  
  const res = await axios.get(`${this.baseUrl}/api/logs`);  
  return res.data;  
}
```

Și tot așa pentru orice rută nouă.

- **Testare:** Implementăm **teste automate** și manuale. În `scripts/` putem crea un PowerShell `qa_tests.ps1` care rulează un set de apeluri curl (ex. crează un lead, marchează conversia, generează un video, apoi verifică prin GET corespunzătoare că rezultatele sunt corecte). De asemenea, folosim unelte de testare (ex. Postman/Newman, sau un framework Jest + axios-mock pentru frontend) pentru a valida integrările.

Figura: Exemplu de flux FE-BE-DB finalizat (din punct de vedere conceptual).

Bibliografie: Descrierea generală și arhitectura platformei AutoPro Daune urmează blueprint-ul oficial, iar implementarea detaliată a fiecărei funcționalități este bazată pe codul real din arhiva proiectului (rutare FastAPI, schema Supabase, logică React/Vite). Citețile de mai sus vin din documentația oficială a proiectului, care subliniază scopul MVP-ului și fluxurile cheie (formular lead, workflow n8n, notificări). Planul de mai sus poate fi aplicat direct în refactorizarea codului existent pentru producție.